

SGD with Adaptive Learning Rate

Kun Yuan

November 11, 2025

In this lecture, we will discuss

- Momentum SGD
- Preconditioned SGD
- AdaGrad and RMSProp
- Adam

Gradient descent can be slow

- Gradient descent can be very slow for ill-conditioned problems
- For example, GD converges very slow when μ/L is sufficiently small¹

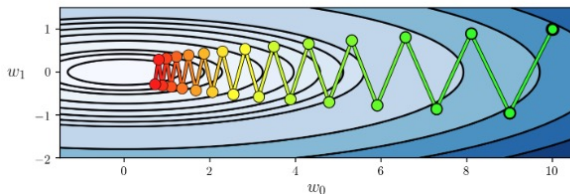


Figure: GD converges slow for ill-conditioned problem

¹Image is from https://github.com/jermwatt/machine_learning_refined

Accelerated gradient descent: algorithm and intuition

- We have to alleviate the “Zig-Zag” to accelerate the algorithm
- Polyak’s momentum gradient descent

$$x_t = x_{t-1} - \gamma \nabla f(x_{t-1}) + \beta(x_{t-1} - x_{t-2})$$

where $\beta \in (0, 1)$ is the momentum parameter

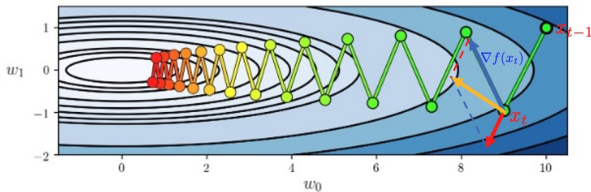


Figure: Momentum can alleviate the “Zig-Zag”

Accelerated gradient descent: algorithm and intuition

- We have to alleviate the “Zig-Zag” to accelerate the algorithm
- Polyak’s momentum gradient descent

$$x_t = x_{t-1} - \gamma \nabla f(x_{t-1}) + \beta(x_{t-1} - x_{t-2})$$

where $\beta \in (0, 1)$ is the momentum parameter

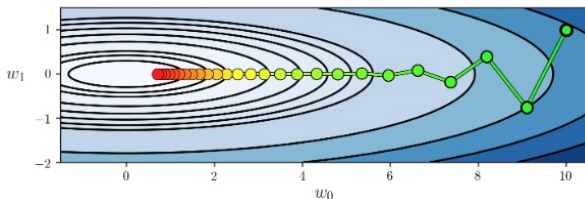


Figure: Momentum can alleviate the “Zig-Zag”

Momentum SGD

- Consider the stochastic optimization problem:

$$\min_{x \in \mathbb{R}^d} f(x) = \mathbb{E}_{\xi \sim D}[F(x; \xi)]$$

- ξ is a random variable indicating data samples
 - D is the data distribution; unknown in advance
 - $F(x; \xi)$ is differentiable in terms of x
-
- Momentum SGD:

$$x_{k+1} = x_k - \gamma \nabla F(x_k; \xi_k) + \beta(x_k - x_{k-1}) \quad (1)$$

where ξ_k is a data realization sampled at iteration k , and $\beta \in (0, 1)$.

Momentum SGD

- Momentum SGD can be rewritten into

$$m_k = \beta m_{k-1} + \nabla F(x_k; \xi_k)$$

$$x_{k+1} = x_k - \gamma m_k$$

where $v_0 = 0$.

- To see it, we have the following recursion from the second line

$$\beta x_k = \beta x_{k-1} - \gamma \beta m_{k-1}.$$

Subtract it from the second line, we have

$$\begin{aligned} x_{k+1} - \beta x_k &= x_k - \beta x_{k-1} - \gamma(m_k - \beta m_{k-1}) \\ &= x_k - \beta x_{k-1} - \gamma \nabla F(x_k; \xi_k). \end{aligned}$$

Regrouping the terms, we achieve Momentum SGD recursion (1).

Preconditioned GD: motivation

- Another way to accelerate GD is to use preconditioning
- Consider an ill-conditioned quadratic problem

$$\min_x x^T Q x + c^T x$$

where Q is a ill-conditioned matrix. GD is slow when solving the problem

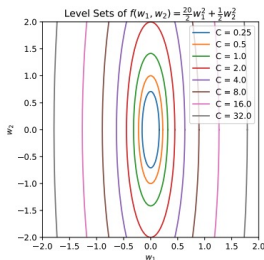


Figure: An ill-conditioned QP problem. (From Prof. Chris De Sa's lecture notes)

Preconditioned GD: motivation

- We now let $x = P^{\frac{1}{2}}w$ for some positive definite matrix P
- Since P is positive definite, x and w is an 1 – 1 mapping
- If we choose $P = Q^{-1}$, we have $x^T Q x = w^T Q^{-\frac{1}{2}} Q Q^{\frac{1}{2}} w = \|w\|^2$
- With $x = P^{\frac{1}{2}}w$ and $P = Q^{-1}$, the ill-conditioned problem becomes

$$\min_w \frac{1}{2} \|w\|^2 + c^T Q^{-\frac{1}{2}} w$$

which is a benign problem. GD is fast to achieve w^* .

- Once w^* is determined, we have $x^* = P^{\frac{1}{2}}w^*$.

Preconditioned GD: motivation

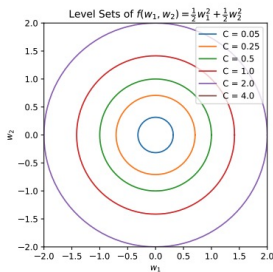
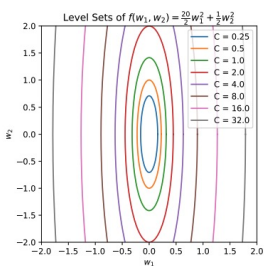


Figure: Left: an ill-conditioned QP problem. Right: a benign QP problem after transformation. (From Prof. Chris De Sa's lecture notes)

Preconditioned GD: derivation

- Consider a general ill-conditioned optimization problem

$$\min_{x \in \mathbb{R}^d} f(x)$$

- We let $x = P^{\frac{1}{2}}w$ so that $g(w) = f(P^{\frac{1}{2}}w)$ is a nice function.
- Use gradient descent to minimize $g(w)$, i.e.,

$$w_{k+1} = w_k - \gamma \nabla g(w_k) = w_k - \gamma P^{\frac{1}{2}} \nabla f(P^{\frac{1}{2}}w_k)$$

- Left-multiplying $P^{\frac{1}{2}}$ to both sides, we achieve

$$\begin{aligned} P^{\frac{1}{2}}w_{k+1} &= P^{\frac{1}{2}}w_k - \gamma P \nabla f(P^{\frac{1}{2}}w_k) \\ \iff x_{k+1} &= x_k - \gamma P \nabla f(x_k) \end{aligned}$$

where P is called the preconditioning matrix.

Preconditioned GD: derivation

- The preconditioned GD algorithm

$$x_{k+1} = x_k - \gamma P \nabla f(x_k)$$

- It is critical to choose the preconditioning matrix P
- If $P = [\nabla^2 f(x_k)]^{-1}$, then preconditioned GD reduces to Newton's method
- It is critical to construct an efficient and effective P matrix

GD v.s. Newton's method

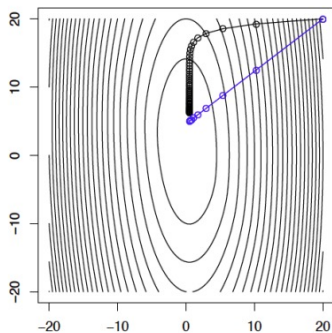


Figure: Convergence comparison between GD and Newton.

Adaptive gradient method (AdaGrad)

- Recall the stochastic optimization problem

$$\min_{x \in \mathbb{R}^d} f(x) = \mathbb{E}_{\xi \sim D}[F(x; \xi)]$$

- Adaptive gradient method

$$g_k = \nabla F(x_k; \xi_k)$$

$$s_k = s_{k-1} + g_k \odot g_k$$

$$x_{k+1} = x_k - \frac{\gamma}{\sqrt{s_k} + \epsilon} \odot g_k$$

where $1/\sqrt{s_k} = \text{col}\{1/\sqrt{s_{k,1}}, \dots, 1/\sqrt{s_{k,d}}\} \in \mathbb{R}^d$ is an element-wise operation, s_0 is initialized as 0, and a small ϵ is added for safe-guard.

Adaptive gradient method (AdaGrad)

- AdaGrad falls into preconditioned SGD
- If we let $P_k = \text{diag}\{\frac{1}{\sqrt{s_{k,1}}+\epsilon}, \dots, \frac{1}{\sqrt{s_{k,d}}+\epsilon}\} \in \mathbb{R}^{d \times d}$, AdaGrad becomes

$$x_{k+1} = x_k - \gamma P_k g_k$$

where P_k is a time-varying preconditioning matrix.

- AdaGrad imposes smaller learning rates for notable gradient directions
- AdaGrad imposes larger learning rates for insignificant gradient directions

Adaptive gradient method (AdaGrad)

AdaGrad alleviates the “Zig-Zag” phenomenon

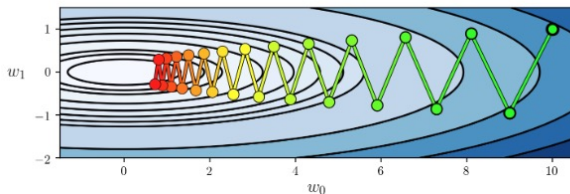


Figure: GD converges slow for ill-conditioned problem

Adaptive gradient method (AdaGrad)

AdaGrad alleviates the “Zig-Zag” phenomenon

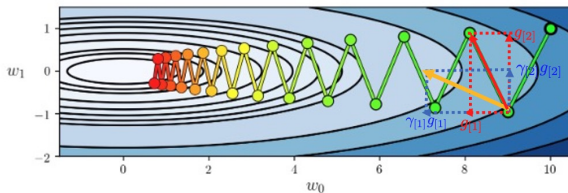


Figure: AdaGrad has alleviated “Zig-Zag” phenomenon

Adaptive gradient method (AdaGrad)

The learning rate in AdaGrad is adaptive; no need to tune.

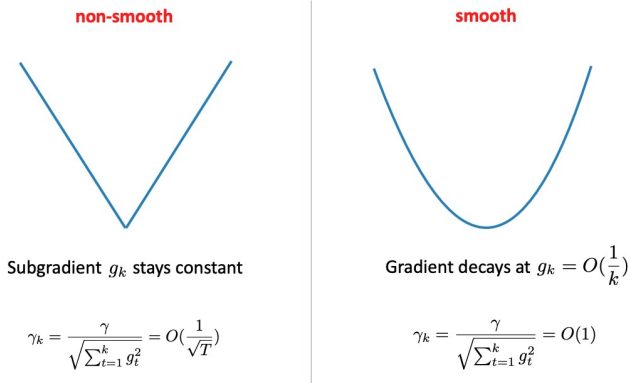


Figure: AdaGrad automatically adapts to problem structure².

²These examples are from <https://conferences.mpi-inf.mpg.de/adfocs/material/alina/adaptive-L1.pdf>

RMSProp

- Since s_k keeps increasing, the rate γ_k in AdaGrad keeps decreasing
- AdaGrad may suffer from slow convergence
- RMSProp proposes a different way to construct s_k

$$s_k = \beta s_{k-1} + (1 - \beta)g_k \odot g_k$$

where $\beta \in (0, 1)$.

- In RMSProp, only the most recent g_k influences the convergence rate

- RMSProp has the following update

$$g_k = \nabla F(x_k; \xi_k)$$

$$s_k = \beta s_{k-1} + (1 - \beta) g_k \odot g_k$$

$$x_{k+1} = x_k - \frac{\gamma}{\sqrt{s_k} + \epsilon} \odot g_k$$

where s_0 is initialized as 0, and a small ϵ is added for safe-guard.

- Adam applies both momentum and adaptive rate to alleviate “Zig-Zag”.

$$g_k = \nabla F(x_k; \xi_k)$$

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

$$s_k = \beta_2 s_{k-1} + (1 - \beta_2) g_k \odot g_k$$

$$x_{k+1} = x_k - \frac{\gamma}{\sqrt{s_k} + \epsilon} \odot m_k$$

where m_0 and s_0 are initialized as 0, and a small ϵ is added for safe-guard.

Animation of different adaptive SGD's

<https://imgur.com/a/Hqolp>